

OOP (Object Oriented Programming)

OOP in Python

Everything is an object.

- The int object 10 is instantiated.
- The variable i is referencing the object.

```
i = 10 # Python
```

This is Java equivalent.

```
Integer i = new Integer(10);
```

| `__init__` as an Initializer

- Object creation in Python is a two-step process:
`__new__` allocates the instance, and
`__init__(self)` initializes its state.
- Since custom classes almost always override `__init__`, it is commonly referred to as the constructor.

```
class Airplane:  
    def __init__(self):  
        self.speed = 0  
        self.altitude = 0  
        self.rollAngle = 0  
        self.pitchAngle = 0
```

- We can use parameters and default values.

```
# when no argument is given,  
# default values are assigned  
def __init__(self, speed = 0,  
              altitude = 0,  
              rollAngle = 0):  
    self.speed = speed  
    self.altitude = altitude  
    self.rollAngle = rollAngle
```

Instantiation

- No `new` keyword needed in Python.
- Calling `Airplane()` executes `__new__` to create the object, then invokes `__init__`.

```
# Creates the instance and calls Airplane.__init__(a)  
a = Airplane()
```

Overriding Python class methods

- By default, printing a user-defined object shows an identity-based representation with its class name and ID (`<Airplane object at 0x...>`).
- Override `__str__` to provide a readable, user-friendly string representation.

```
class Airplane:
    def __init__(self, speed = 0):
        self.speed = speed

    def __str__(self):
        return f"Airplane speed: {self.speed} km/h"

b = Airplane(speed = 100)
print(b) # Output: Airplane speed: 100 km/h
```

Overriding `__eq__`

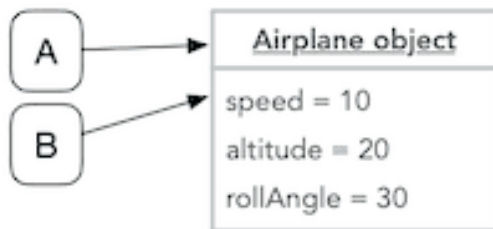
- The `==` operator delegates to `__eq__()`; by default, `object.__eq__()` compares reference identity (like `is`).
- Override `__eq__` to customize value comparison logic instead.

```
class Airplane:
    def __init__(self, speed = 0):
        self.speed = speed

    def __eq__(self, other): # compare speed values
        if isinstance(other, Airplane):
            return self.speed == other.speed
        return False
```

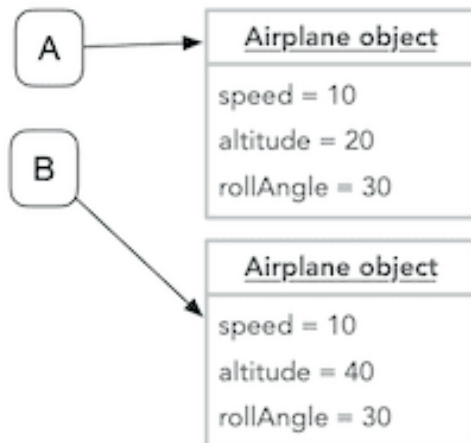
Shallow and Deep Copy

- In OOP, the `=` (assignment) operator copies the reference, not the object — both names end up pointing at the same object.



```
1 a = Airplane(speed = 100)
2 b = a
3 print(a == b) # True as same reference
4 b.speed = 500
5 print(a.speed) # 500
```

- To make a new, independent object, use `copy.deepcopy()`, which also copies any nested mutable objects. Use `copy.copy()` for a shallow copy (new outer object, still-shared nested objects). A few objects customize this via `__deepcopy__`.



```
1 from copy import deepcopy
2 ...
3 a = Airplane(speed = 100)
4 b = deepcopy(Airplane(speed = 100))
5 print(a == b) # False
6 b.speed = 500
7 print(a.speed) # 100
```

Python and Interface

- Python does not have a dedicated **interface** keyword like Java, but supports interface contracts:
 - **Abstract Base Classes (**abc.ABC**)**:
Explicit contracts with **@abstractmethod**
 - **Protocols (**typing.Protocol**)**: Structural subtyping (static duck typing)
 - **Duck Typing**: Implicit contract based on method signatures



```
1 class Animal(object): # Normal class
2     def makeSound(): print('')
3 class FourLegged(object): # Interface
```

Python Duck Typing

- Duck typing means Python cares about an object's behavior, not its class hierarchy.
- If an object has the required methods/attributes, it can be used regardless of its type.
- "If it walks like a duck and quacks like a duck, it's a duck."

- Both list and tuple implement `__len__`, so the built-in `len()` function works on both seamlessly:

```
# len works for both via duck typing:  
a = []  
print(len(a)) # calls a.__len__()  
  
b = ()  
print(len(b)) # calls b.__len__()
```

- Both `Duck` and `Tiger` provide a `quack()` method:

```
class Duck:
    def quack(self):
        print("Quack")
```

```
class Tiger:
    def quack(self):
        print("Roar")
```

```
def quack_checker(q):
    q.quack()
```

```
quack_checker(Duck())
quack_checker(Tiger())
```

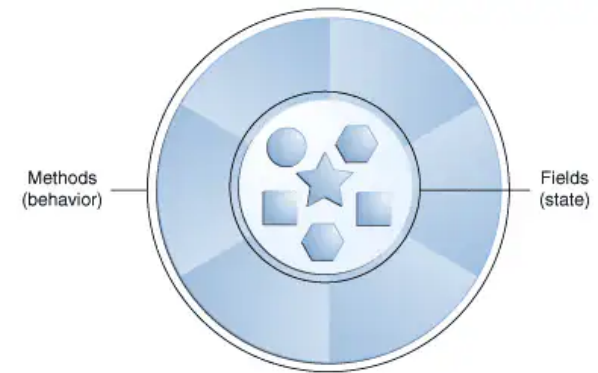
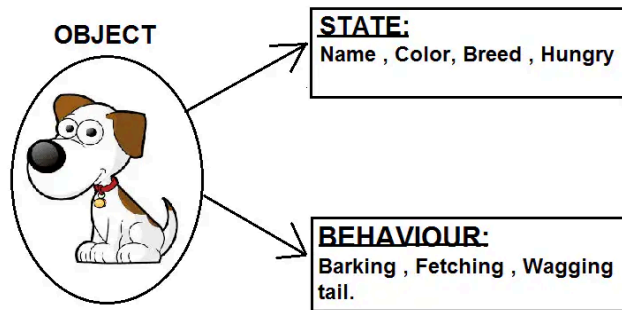
Python: Modeling with OOP

Python is an OOP language that can model real-world concepts effectively.

- Abstraction
- Polymorphism
- Inheritance
- Encapsulation

Abstraction

- Software modeling captures essential real-world features while ignoring irrelevant details.
- This is called abstraction.
- Abstraction models an object's states (nouns) and behaviors (verbs).



- A Dog object has states and behaviors.
- In software design, states are modeled as attributes and behaviors as methods.

| Polymorphism

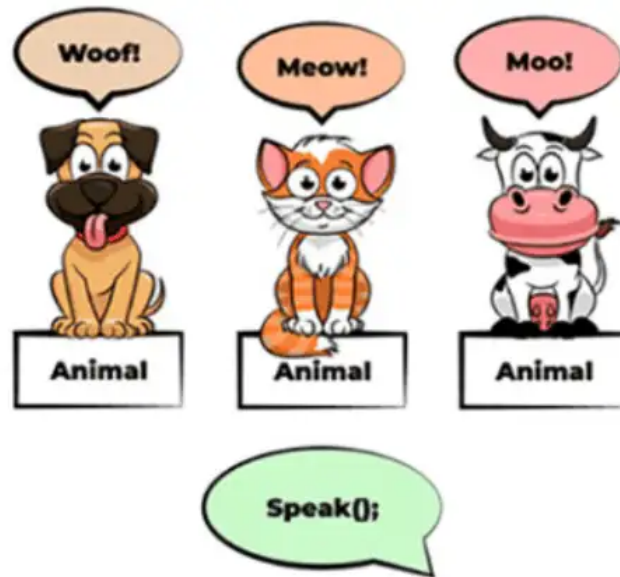
- **One interface, different implementations:** The same method call executes different behaviors depending on the calling object's class.
- Polymorphism allows uniform treatment of different types through a shared interface.

```
class Dog:
    def speak(self):
        return "Woof! Woof!"

class Cat:
    def speak(self):
        return "Meow!"

def make_sound(animal):
    print(animal.speak()) # Polymorphic call
```

```
make_sound(Dog()) # Woof! Woof!  
make_sound(Cat()) # Meow!
```



Inheritance

- We can model the real world with hierarchical general and specific concepts.
- This is called inheritance: superclasses are general, subclasses are specific, and creating one is called subclassing.

Super Class



Animals

Child Class



Amphibians



Reptiles



Mammals



Birds

| Encapsulation

- Encapsulation bundles internal data (state) and methods together, controlling access to internal state.

- In Python:
 - `_variable`: Protected by convention (internal use).
 - `__variable`: Private via name mangling (`_Class__variable`).
 - `@property`: Controlled getter/setter access.

